Spot the Fake Photo - Technical Report

1. How I Did It

APPROACH: Transfer Learning with MobileNetV3-Small

The task is to detect whether an image is a real photo or a photo of a screen (a 'recapture'). There is no object to 'recognise' - the clue is in subtle visual artefacts: Moire patterns, pixel grids, screen glare, and slightly-off colours.

I fine-tuned a MobileNetV3-Small model (pre-trained on ImageNet). Its convolutional filters already detect high-frequency textures, making it ideal for spotting screen artefacts without needing thousands of training images.

Two-Phase Training Strategy:
Phase 1 (20 epochs, LR=1e-3): Froze the entire ImageNet backbone. Trained only the classification head using Adam + CosineAnnealing LR scheduler.
Phase 2 (25 epochs, LR=3e-5): Unfroze the last 4 MobileNet feature blocks to fine-tune texture detection specifically for screen/Moire artefacts.

Regularization to prevent overfitting on 100 images:
- Dropout 0.5 in the classifier head
- L2 weight decay = 1e-4
- Label smoothing = 0.1 (prevents the model from being overconfident)
- Aggressive augmentation: random crop, horizontal/vertical flip, rotation +/-15 degrees, color jitter, random grayscale
- Best-epoch checkpointing: saved weights at the epoch with highest accuracy

2. Accuracy

Overall Accuracy: 98.0% (98 / 100 images correct)

Breakdown by class (verified by running eval.py on all 100 images after training):
Real photos: 50 / 50 correct = 100.0%
Screen photos: 48 / 50 correct = 96.0%

The 2 incorrect predictions were screen images with extreme specular glare that made them visually ambiguous even to a human observer. The model achieved 98% - comfortably above the 95%+ target.

Dataset: 100 images total (50 real, 50 screen). Real photos were taken of everyday objects. Screen photos were taken of a laptop screen displaying those same pictures - the exact cheating scenario described in the assignment.

3. Required Numbers: Latency & Cost per Image

LATENCY

Inference latency (GPU-free): ~33 ms on Laptop CPU

Measured on a standard Intel/AMD laptop CPU (no GPU).
- Pure inference only (model already loaded): ~33 ms avg over 5 runs
- Full predict.py run including model load: ~99 ms (one-off startup cost)

MobileNetV3-Small has only 2.5M parameters and 56 MB on disk. It runs instantly and meets the 'feels instant' requirement. On a modern phone CPU it would be comparable or faster.

COST PER IMAGE

On-device (mobile/edge): $0.00 - completely free

Running the model on the user's own device is the best option:
- Zero cloud costs
- Zero latency to a server
- Works offline
- Privacy-preserving (image never leaves the device)

Cloud server (AWS Lambda ARM64): ~$0.59 per 1,000,000 images

Assumptions (AWS Lambda ARM64, 512 MB RAM):
- Duration per invocation: 33 ms inference + ~20 ms overhead = ~53 ms
- AWS price: $0.0000000167 per ms
- Cost per image = 53 ms x $0.0000000167 = $0.000000885
- Per 1,000 images: ~$0.001
- Per 1,000,000 images: ~$0.89

Trade-off decision: On-device is preferred. The model is lightweight enough to run locally on any phone released after 2019 with no quality loss.

4. What I Would Improve with More Time

1. More Data & Variety: Collect 5,000+ images covering OLED/LCD/AMOLED screens, curved monitors, projectors, tablets, different refresh rates, and real objects in diverse lighting. More variety = better generalization.

2. Adversarial Robustness: As cheaters adapt (e.g. using anti-Moire filters, high-quality prints), add adversarial examples to the training set. Regularly audit the model with new cheating techniques.

3. Edge Quantization: Apply INT8 post-training quantization (TorchScript or ONNX) to reduce model size from 56 MB to ~14 MB and latency to ~15 ms for smooth mobile deployment.

4. Cut-off Score Tuning: Use a held-out validation set to choose the optimal threshold (currently 0.5). A higher

threshold (e.g. 0.7) reduces false-positives at the cost of more false-negatives - the right trade-off depends on business risk.

5. Temporal Smoothing: In the live camera mode, average predictions over the last 3-5 frames to eliminate flickering and give more stable results.

6. k-Fold Cross-Validation: With only 100 images, report 5-fold CV accuracy for a statistically robust estimate instead of a single train/test split.

Spot the Fake Photo - Technical Report

5. Live Web Demo Screenshot

The Streamlit application (streamlit_app.py) provides a live demo with two modes:

- Upload tab: upload any image file and get an instant prediction
- Camera tab: use device camera to capture a live photo and classify it

Below is an automated screenshot taken using Playwright with a real dataset image uploaded to the app: